

Assignment 4: Comparative Study of Isolation Mechanisms (Accessing Services Across VPCs in AWS)

ComS/CprE 4590X/5590 2026 Spring

Total: 30 points

Due: **Wednesday, March 25, 11:59 PM**

Objective

To study the foundational isolation mechanisms through comparison, this lab explores three mechanisms for accessing services across AWS VPCs:

- Access via the **public Internet**
- Access using **AWS PrivateLink**
- Access using **VPC Peering**

Each task uses an independent environment.

Important: After finishing a task, terminate resources that are no longer needed to avoid unnecessary AWS charges.

Total points: **30**

Submission

Submit a PDF report containing:

- Required screenshots
- Command outputs
- Recorded IP addresses

Label each screenshot with the corresponding step number.

Note: This document provides the main but not all steps. You should figure out the absent steps based on the knowledge and skills learned in class and prior assignments.

1 Task 1: Access Service via Internet Across Regions (6 points)

In this task, a client instance in one region accesses a web server in another region through the Internet.

Step 1: Create VPC in Region A (2 point)

Choose a region (e.g., **us-east-1**).

Create a VPC:

- Name: Internet-VPC-A
- CIDR: 10.0.0.0/16

Create a public subnet:

- CIDR: 10.0.1.0/24

Create and attach an Internet Gateway.

Update the route table:

Destination: 0.0.0.0/0

Target: Internet Gateway

Snapshot

VPC Console → **Your VPCs** → select **Internet-VPC-A** → open the **Resource map** tab showing the VPC, subnet, route table, and Internet Gateway.

Step 2: Launch Web Server (1 points)

Launch an EC2 instance:

- AMI: Amazon Linux
- Instance type: t2.micro
- VPC: Internet-VPC-A
- Subnet: 10.0.1.0/24
- Auto-assign Public IP: Enable

Security group rules:

- SSH (22) from your IP
- HTTP (80) from Anywhere

SSH to the instance and install the web server:

```
sudo yum update -y
sudo yum install httpd -y

echo "Hello from Region A server" \
| sudo tee /var/www/html/index.html

sudo systemctl start httpd
sudo systemctl enable httpd
```

Snapshot

EC2 Console → **Instances** → select the server instance → **Details tab** showing:

- Public IPv4 address
- VPC ID
- Subnet ID

Also include a terminal screenshot showing the successful installation and start of the `httpd` service.

Step 3: Create Client VPC in Region B (2 point)

Switch to another region (e.g., **us-west-2**).

Create:

- VPC: Internet-VPC-B
- CIDR: 10.1.0.0/16
- Subnet: 10.1.1.0/24

Attach an Internet Gateway.

Launch a client EC2 instance with a public IP.

Allow SSH from your IP.

Snapshot

EC2 Console → **Instances** → select the client instance → **Details tab** showing:

- Region
- Public IPv4 address
- VPC ID

Step 4: Test Internet Access (1 points)

From the client instance:

```
curl http://<server-public-ip>
```

Expected output:

Hello from Region A server

Snapshots

- Terminal window on the client instance showing the `curl` command and output
- EC2 Console → server instance → **Details tab** clearly showing the server's Public IPv4 address used in the test

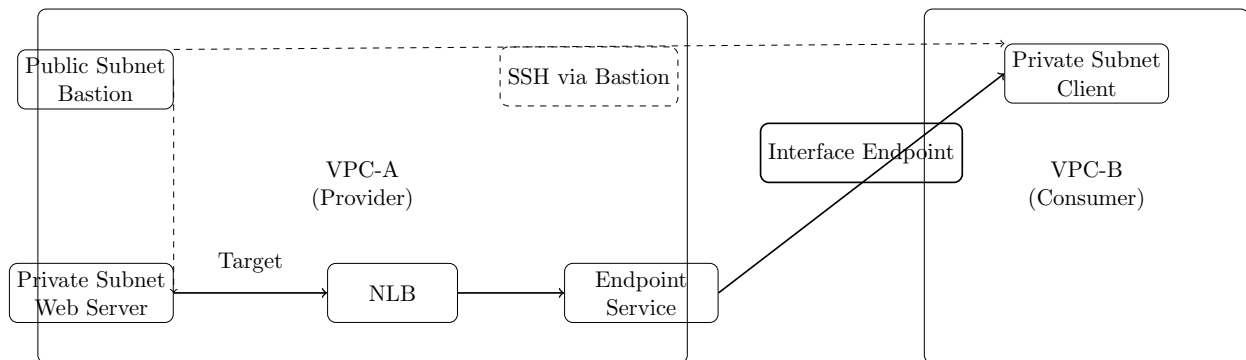
Cleanup suggestion

After completing this task, terminate both EC2 instances and delete the VPCs if they are not needed later.

2 Task 2: Access Service Using AWS PrivateLink (12 points)

In this task, the service runs in a **private subnet**. Access is provided through AWS PrivateLink.

Architecture Diagram



Step 1: Create VPC-A with Private Subnets (2 point)

Create VPC-A:

- CIDR: 10.10.0.0/16

Create a subnet at each availability zone. These subnets should include:

- Public subnet: 10.10.1.0/24
- Private subnet: 10.10.2.0/24
- Other public or private subnets (choose appropriate CIDR blocks)

Attach an Internet Gateway.

Launch a **bastion host** in a public subnet.

Snapshot

VPC Console → **Your VPCs** → select VPC-A → **Resource map** tab showing:

- Public subnet
- Private subnet
- Internet Gateway

Step 2: Launch Server in a Private Subnet (2 points)

Launch an EC2 instance:

- VPC: VPC-A
- Subnet: private subnet
- No public IP

Security group:

- Allow HTTP from VPC-A CIDR
- Allow SSH from bastion host

Install the web server.

Snapshot

EC2 Console → **Instances** → select the server instance → **Details tab** showing:

- Private IPv4 address
- VPC ID
- Subnet ID
- **Public IPv4 address field showing none**

Step 3: Create Network Load Balancer (3 points)

Create a Network Load Balancer with listener TCP port 80.

Attach the server instance through a target group.

Verify target status is:

Healthy

Snapshot

- EC2 Console → **Target Groups** → select the target group → **Targets tab** showing the server instance with status **Healthy**.
- EC2 Console → **Load Balancers** → select the network load balancer → **Overview tab** showing the key information of the NLB.

Step 4: Create Endpoint Service (2 point)

Navigate to:

VPC → **Endpoint Services**

Create endpoint service associated with the NLB.

Enable:

- Acceptance required

Snapshot

VPC Console → **Endpoint Services** → select the created service → **Details tab** showing:

- Service name
- Associated Network Load Balancer

Step 5: Create VPC-B and Interface Endpoint (2 point)

Create VPC-B:

- CIDR: 10.20.0.0/16

Create subnet and launch a client instance.

Create an **Interface Endpoint** using the service name.

Accept the request in VPC-A.

Snapshot

VPC Console → **Endpoints** → select the interface endpoint → **Details tab** showing:

- Endpoint status: Available
- DNS name

Step 6: Test Connectivity (1 point)

From the client instance:

```
curl http://<endpoint-dns>
```

Expected output:

```
Hello from PrivateLink server
```

Snapshot

Terminal on the client instance showing the `curl` command and output.

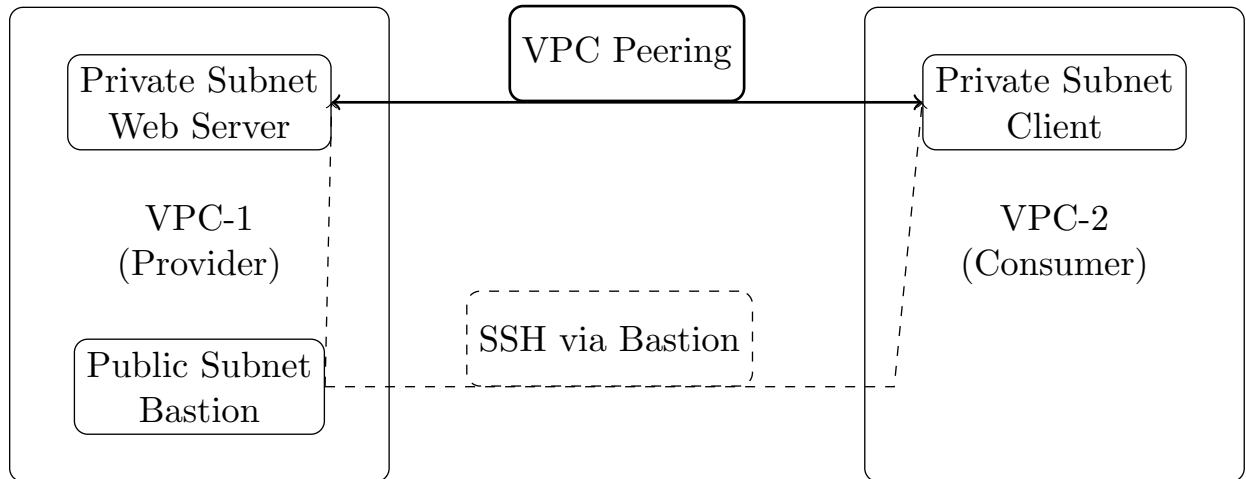
Cleanup suggestion

Terminate EC2 instances and delete endpoint services when finished.

3 Task 3: Access Service Using VPC Peering (12 points)

In this task, the service is located in a **private subnet**.

Architecture Diagram



Step 1: Create Two VPCs (2 point)

Create:

- VPC-1: 10.30.0.0/16
- VPC-2: 10.40.0.0/16

Each VPC should contain:

- one public subnet
- one private subnet

Launch a bastion host in the public subnet of VPC-1.

Snapshot

VPC Console → **Your VPCs** page showing both VPC-1 and VPC-2.

Step 2: Launch Server and Client (2 points)

Server:

- VPC-1 private subnet
- No public IP

Client:

- VPC-2 public subnet

Install web server on the server instance.

Snapshot

EC2 Console → **Instances** page showing both instances and their **VPC IDs**.

Step 3: Create VPC Peering Connection (3 point)

Navigate to:

VPC → Peering Connections

Create connection:

- Requester: VPC-1
- Acceptor: VPC-2

Accept the request.

Snapshot

VPC Console → **Peering Connections** page showing the connection with status **Active**.

Step 4: Update Route Tables (4 point)

Add routes.

VPC-1 route table:

Destination: 10.40.0.0/16

Target: Peering connection

VPC-2 route table:

Destination: 10.30.0.0/16

Target: Peering connection

Snapshot

VPC Console → **Route Tables** → select the route table → **Routes tab** showing the added peering route.

Step 5: Test Connectivity (1 point)

From the client instance:

```
curl http://<server-private-ip>
```

Expected result:

```
Hello from Peering server
```

Snapshot

Terminal on the client instance showing the `curl` command and output.

Cleanup suggestion

Delete the VPC peering connection and terminate instances.

Additional References

The following AWS documentation may be consulted:

- AWS PrivateLink Getting Started Guide:
<https://docs.aws.amazon.com/vpc/latest/privatelink/getting-started.html>
- What is AWS PrivateLink:
<https://docs.aws.amazon.com/vpc/latest/privatelink/what-is-privatelink.html>
- Create a VPC Peering Connection:
<https://docs.aws.amazon.com/vpc/latest/peering/create-vpc-peering-connection.html>
- VPC Peering Concepts:
<https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-basics.html>